

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Student Name & Reg. No.	Guru Divya Darshini U (Reg. No. 24011101032)	

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

Observed setup (my run)

- I loaded the raw inertial-signal files of both UCI HAR partitions and pooled them: 10,299 windows, tensor shape (10299, 128, 9).
- Instead of taking the “first” windows, I drew a **stratified subset of $N = 3000$ windows** (so all six classes stay represented in their natural, roughly balanced proportions) and split it 70/15/15 with stratification: 2100/450/450 windows.
- Normalization: per-channel mean and standard deviation from the **training split only**, applied to validation and test.
- RNN, LSTM and GRU use the same data, seed (42), optimizer, batch size, epochs and evaluation protocol. Software: TensorFlow 2.20.0.
- No GPU was detected in this run (the GPU list printed was empty), so **every training time in this record is a CPU time**. I trained each configuration once, so small differences between models should be read with that in mind.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.

5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

Training : (N_train, 128, 9)

Validation : (N_val, 128, 9)

Testing : (N_test, 128, 9)

Number of classes: 6

Number of features per time step: 9

Sequence length: 128

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

Results: preprocessing output

Pooled raw data: (10299, 128, 9) labels: (10299,)

Input tensor shape:

Training : (2100, 128, 9)

Validation : (450, 128, 9)

Testing : (450, 128, 9)

Number of classes: 6

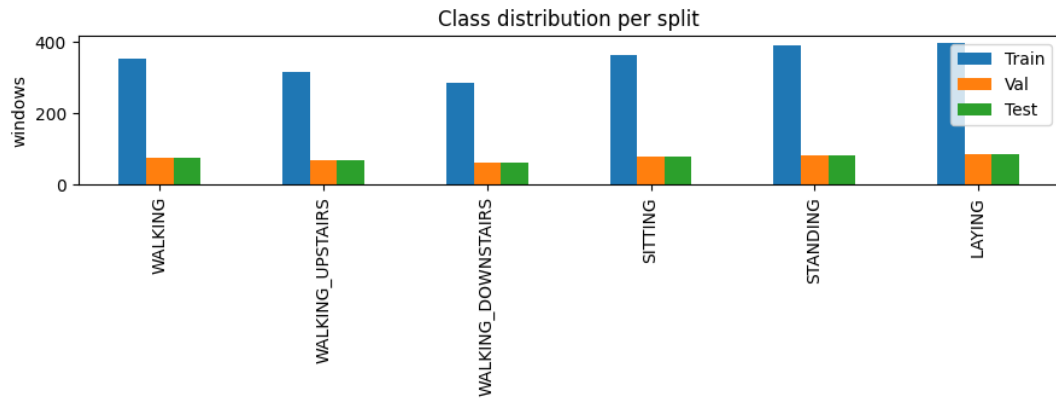
Number of features per time step: 9

Sequence length: 128

Class distribution check (windows per split):

Class	Train	Validation	Test
WALKING	351	76	75
WALKING_UPSTAIRS	315	68	67
WALKING_DOWNSTAIRS	286	61	62
SITTING	363	77	78
STANDING	389	83	83
LAYING	396	85	85
Total	2100	450	450

The largest training class (LAYING, 396) is only about $1.4\times$ the smallest (WALKING_DOWNSTAIRS, 286), so the classes are roughly balanced, and validation and test keep the same class proportions as training because the splits were stratified.



Class distribution per split (2100 / 450 / 450 windows).

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

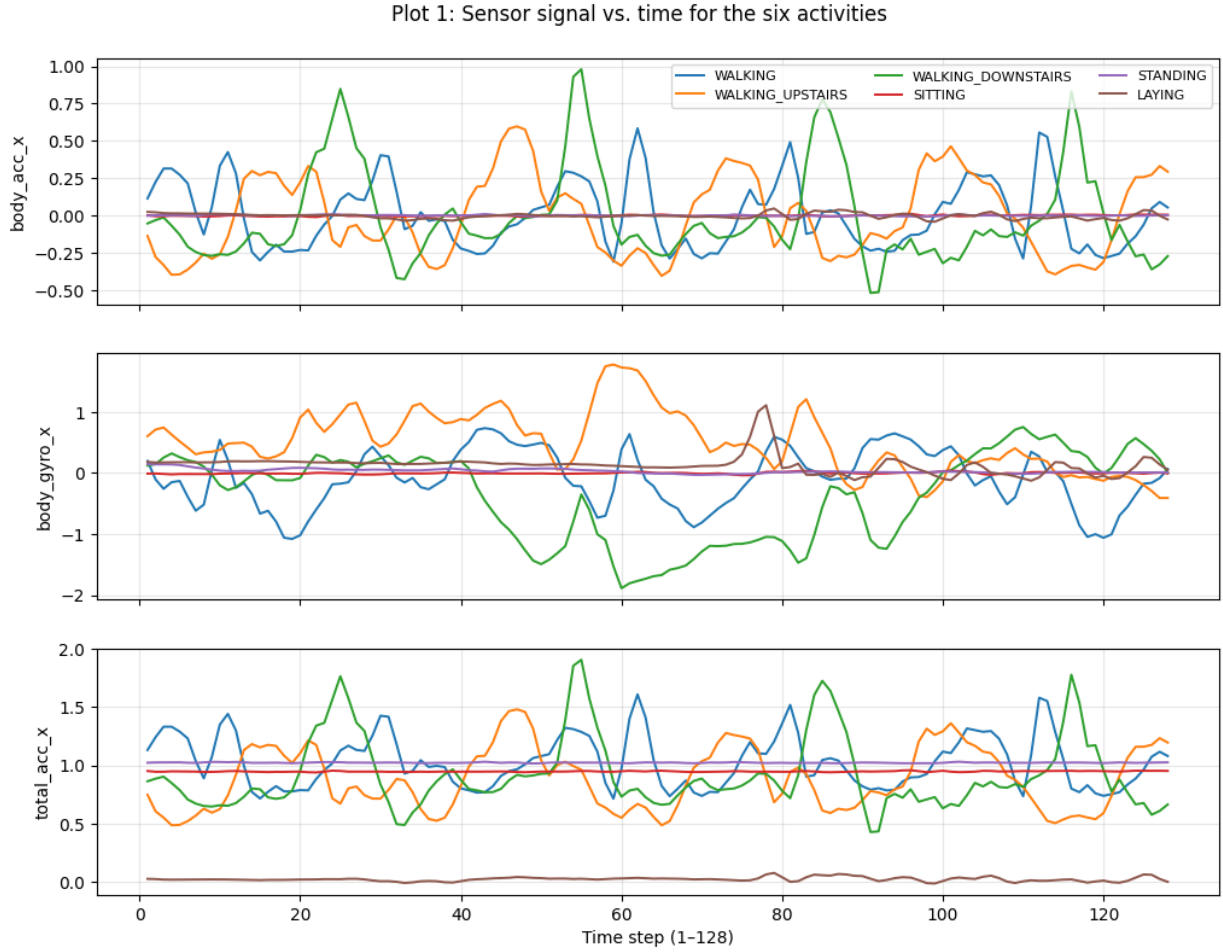
- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

Students should observe that temporal patterns differ across activities.

Required inference:

1. What temporal pattern is visible?
2. Which activities exhibit similar patterns?
3. Why is the ordering of measurements important?

Results: Plot 1 and inference



Plot 1: Sensor signal versus time step (1–128) for body_acc_x, body_gyro_x and total_acc_x, one training window from each of the six activities.

To back up what the plot shows with numbers, I computed the standard deviation of each window over time and averaged it per class (training windows):

Class	mean temporal std, body_acc (xyz)	mean temporal std, body_gyro (xyz)	mean total_acc_x
WALKING	0.1749	0.4274	0.9969
WALKING_UPSTAIRS	0.1907	0.4446	0.9537
WALKING_DOWNSTAIRS	0.2277	0.4844	0.9949
SITTING	0.0102	0.0212	0.9558
STANDING	0.0110	0.0338	1.0018
LAYING	0.0116	0.0246	0.0654

Inference.

- Temporal pattern.** The three walking activities produce large, repeating oscillations (in the plotted windows body_acc_x swings roughly between -0.5 and $+1.0$, and body_gyro_x between about -2 and $+1.8$), while SITTING, STANDING and LAYING are almost flat lines. The within-window standard deviation of body acceleration is 0.17 – 0.23 for walking versus 0.010 – 0.012 for the static classes, i.e. roughly 15 to 22 times larger.
- Similar activities.** The three walking variants look alike (periodic gait cycles, differing mostly in amplitude, with WALKING_DOWNSTAIRS the strongest), and SITTING and STANDING look alike (both nearly motionless; in total_acc_x they differ only by a small offset, about 0.96 versus 1.00). LAYING is also flat, but its total_acc_x is about 0.07 instead of about 1 , because gravity acts along a different sensor axis when the body is horizontal, so it is easy to separate.
- Why order matters.** What identifies a walking window is the *sequence* of rises and falls, i.e. the gait period. If I shuffled the 128 time steps of a window the same values would still be there, but the periodic structure would be gone and it would look like noise. A model therefore has to respect the order of measurements, which is exactly what a recurrent network does through its hidden state.

7. Vanilla RNN Architecture

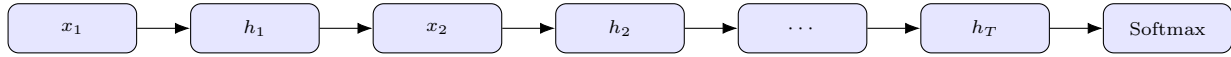
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

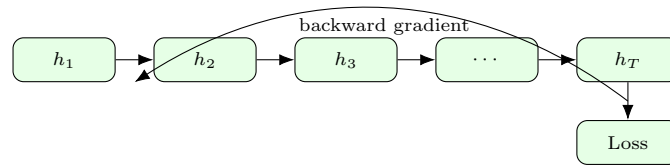
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

Results: numerical exercise

Hand calculation with $h_t = \tanh(0.5x_t + 0.8h_{t-1} + 0.1)$ and $h_0 = 0$:

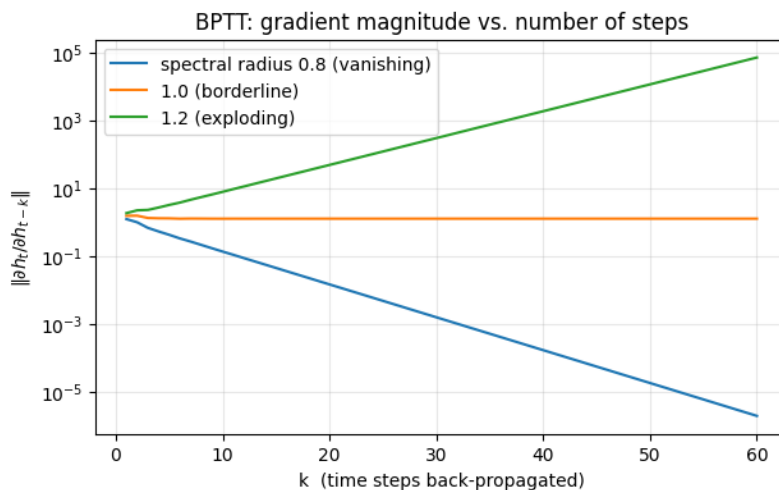
$$\begin{aligned} t = 1 : \quad & 0.5(0.5) + 0.8(0) + 0.1 = 0.3500 & \Rightarrow h_1 = \tanh(0.3500) = 0.3364 \\ t = 2 : \quad & 0.5(0.7) + 0.8(0.3364) + 0.1 = 0.7191 & \Rightarrow h_2 = \tanh(0.7191) = 0.6164 \\ t = 3 : \quad & 0.5(0.2) + 0.8(0.6164) + 0.1 = 0.6931 & \Rightarrow h_3 = \tanh(0.6931) = 0.6000 \end{aligned}$$

Comparison with the program (NumPy loop, and a Keras `SimpleRNN` with one unit whose weights I set to $W_x = 0.5$, $W_h = 0.8$, $b = 0.1$):

	Manual / NumPy	Keras SimpleRNN	difference
h_1	0.336376	0.336376	0.0
h_2	0.616352	0.616352	0.0
h_3	0.599958	0.599958	0.0

The program printed `Match: True`, so the manual values agree with the program. I also noticed h_2 and h_3 are close (0.616 and 0.600) even though x_3 is much smaller than x_2 : the state carries memory of the earlier input ($0.8h_2$ contributes 0.49 to the pre-activation of step 3, versus only 0.1 from x_3).

Results: vanishing and exploding gradients (BPTT illustration)

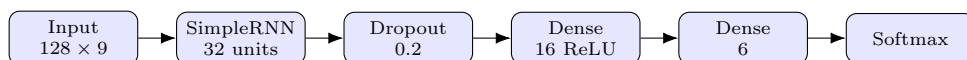


Gradient magnitude $\|\partial h_t / \partial h_{t-k}\|$ versus the number of steps k for a linearized recurrent Jacobian W_h^k with spectral radius 0.8, 1.0 and 1.2.

With spectral radius 0.8 the norm drops from about 1 to roughly 10^{-6} after 60 steps (vanishing); at 1.0 it stays flat; at 1.2 it grows to roughly 10^5 (exploding) (values read off the log-scale plot). With a real tanh the derivative factor $1 - h_t^2 \leq 1$ only makes shrinkage more likely, so a vanilla RNN is pushed towards the vanishing side, and long-range information cannot be learned through 128 steps of repeated multiplication.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

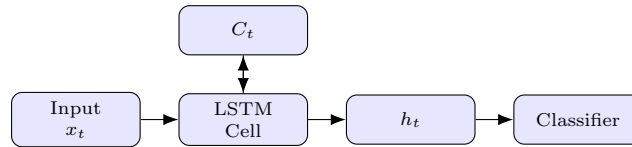
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

Inference: cell state and gates

The key line is $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$. The cell state is updated *additively* and gated element-wise, instead of being pushed through a tanh and a full matrix multiplication at every step as in a vanilla RNN.

- **Retain:** if $f_t \approx 1$ and $i_t \approx 0$, then $C_t \approx C_{t-1}$ and the stored value passes through unchanged. In the backward pass $\partial C_t / \partial C_{t-1} = f_t$, so the gradient along the cell-state path is multiplied by a number close to 1 rather than by W_h and $\tanh'$ at each step, which is what fights vanishing gradients.
- **Discard:** if $f_t \approx 0$, the old memory is erased, and i_t decides how much of the new candidate $\tilde{C}_t$ is written in.
- **Expose:** o_t decides how much of $\tanh(C_t)$ is shown to the next layer as h_t , so information can stay stored without influencing the output yet.

In my experiment the effect is visible: the LSTM reached 90% validation accuracy at epoch 18, while the vanilla RNN never reached 90% in 30 epochs (Section 13).

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

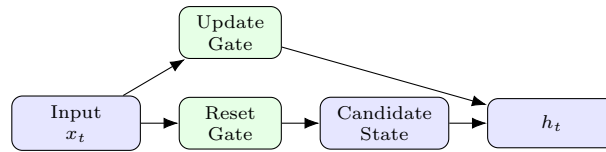
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

Comparison of gates and state representations

	RNN	LSTM	GRU
State(s) carried between steps	h_t	h_t and C_t	h_t
Gates	none	3 (forget, input, output)	2 (update, reset)
Weight blocks in the layer	1	4 (f, i, o, $\tilde{C}$)	3 (z, r, $\tilde{h}$)
Recurrent-layer parameters (32 units, 9 inputs)	1,344	5,376	4,128

The parameter counts follow the block structure. One block has $32(9 + 32) + 32 = 1344$ weights, so the LSTM has $4 \times 1344 = 5376$. Keras' GRU uses separate input and recurrent biases, so its three blocks have $3(32 \cdot 9 + 32 \cdot 32 + 2 \cdot 32) = 4128$, i.e. $3.07 \times$ the RNN layer instead of exactly $3 \times$. These numbers match the model summaries printed by the notebook.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

Results: model sizes and training logs

Model	Recurrent layer	Dense(16)	Dense(6)	Total trainable
RNN (SimpleRNN, 32)	1,344	528	102	1,974
LSTM (32)	5,376	528	102	6,006
GRU (32)	4,128	528	102	4,758

Training log at selected epochs (accuracies in %; Adam, learning rate 10^{-3} , batch 32, 30 epochs):

Model	Epoch	Train loss	Train acc	Val loss	Val acc
RNN	1	1.5387	41.14	1.2756	51.56
RNN	10	0.6652	67.24	0.6253	68.00
RNN	20	0.5550	75.90	0.5048	77.78
RNN	30	0.4979	81.95	0.4604	82.00
LSTM	1	1.5798	36.48	1.3526	41.33
LSTM	10	0.5179	82.14	0.5037	81.33
LSTM	20	0.2128	91.24	0.2428	91.11
LSTM	30	0.2237	91.81	0.2399	90.89
GRU	1	1.5943	37.10	1.3990	51.56
GRU	10	0.2948	90.52	0.2265	93.56
GRU	20	0.1465	94.52	0.1194	96.22
GRU	30	0.1285	95.33	0.1160	95.56

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

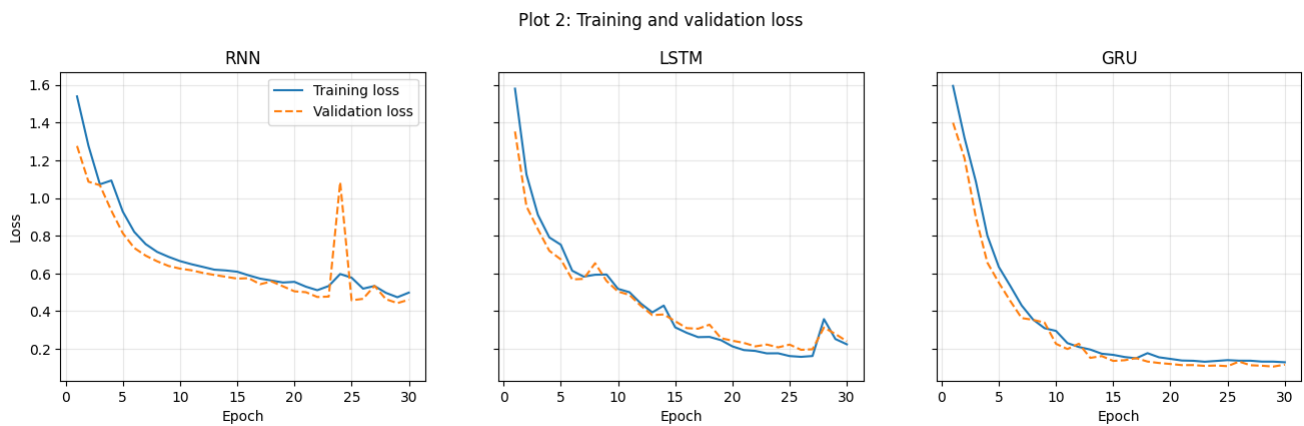
- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

Plot 3: Training and Validation Accuracy

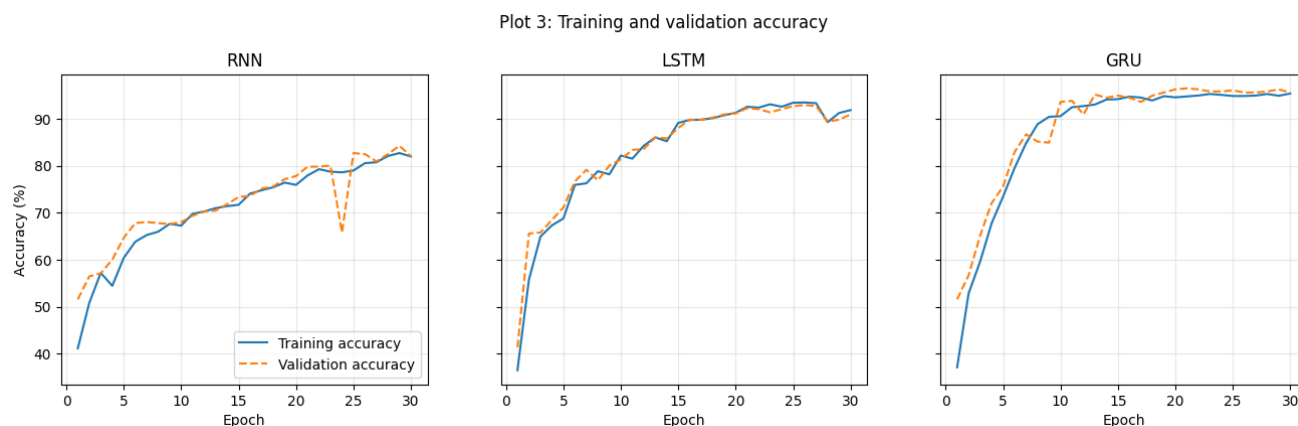
- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

Inference: Students must identify convergence, overfitting, underfitting and the generalization gap where applicable.

Results: Plots 2 and 3



Plot 2: Training and validation loss per epoch for RNN, LSTM and GRU.



Plot 3: Training and validation accuracy (%) per epoch for RNN, LSTM and GRU.

Inference: convergence, over/underfitting, generalization gap

Model	Final train/val loss	Final train/val acc (%)	Gap (train–val, pts)	Best val-loss epoch	First epoch val acc $\geq 90\%$
RNN	0.498 / 0.460	82.0 / 82.0	≈ 0.0	29	never
LSTM	0.224 / 0.240	91.8 / 90.9	+0.9	26	18
GRU	0.129 / 0.116	95.3 / 95.6	−0.2	29	10

- **RNN.** Loss falls from 1.539 to 0.498 (validation 1.276 to 0.460) and validation accuracy from 51.6% to 82.0%, but both curves are still moving at epoch 30 and it never gets to 90%. So it has *not fully converged* within 30 epochs. There is also a sharp spike around epoch 24 (validation loss jumps to roughly 1.1 and validation accuracy dips to roughly 66%) before it recovers. That kind of instability is what I would expect from a plain recurrent layer, although I did not measure gradient norms in the training run.
- **LSTM.** It gets to 90% validation accuracy at epoch 18 and finishes at 91.8% train / 90.9% validation. There is a small bump at epoch 28 (training loss rises to about 0.36 and validation accuracy dips to about 89%), so it is still a little noisy at the end.
- **GRU.** Fastest and smoothest: 93.6% validation accuracy already at epoch 10, a peak of 96.2% at epoch 20, and the loss curves are flat over the last epochs, so it has converged (final 95.3% train / 95.6% validation).
- **Overfitting or underfitting.** None of the three shows classic overfitting (training accuracy far above validation accuracy): the largest gap is +0.9 points for the LSTM. The RNN is best described as *underfitting* in this setting, because its training accuracy itself is only 82% and still rising. Validation loss is often slightly *below* training loss for the GRU because dropout is active only during training and training metrics are averaged over the epoch while the weights are still improving. The notebook’s automatic check flagged the LSTM as “overfitting”; I do not agree with that label, since it was triggered by the epoch-28 bump lifting the final validation loss above its minimum at epoch 26, while train and validation accuracy stay within 1 point.
- **Conclusion.** On the same data and settings the gated models learn much faster than the vanilla RNN (epochs to 90% validation accuracy: GRU 10, LSTM 18, RNN never), and generalization gaps are small for all three because the dropout, the small model and the 2100-window training set keep them from memorizing.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	81.11	90.89	94.89
Macro Precision (%)	82.47	91.54	95.37
Macro Recall (%)	80.30	91.11	95.14
Macro F1 (%)	80.43	91.13	95.18
Parameters	1,974	6,006	4,758
Training Time (s)	48.7	96.3	144.6

All values are measured on the 450-window test set (test used only once, for the final evaluation). Training times are CPU times from a single run each. One test window is worth $1/450 = 0.22$ accuracy points, so the GRU–LSTM accuracy difference of 4.0 points corresponds to 18 windows (427 versus 409 correct).

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

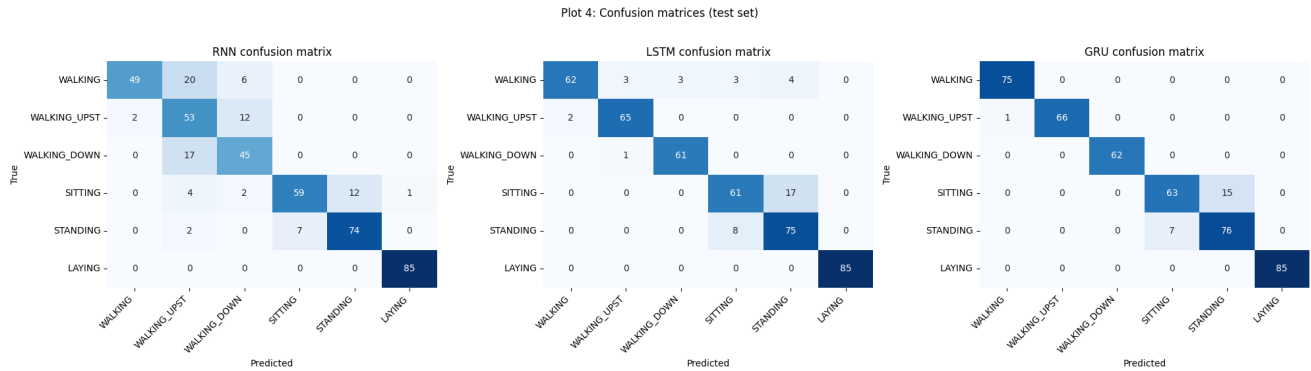
Generate a separate confusion matrix for each model.

Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

The inference must be based on the student’s observed matrix rather than on an assumed result.

Results: Plot 4 and inference



Plot 4: Confusion matrices on the test set (rows = true class, columns = predicted class).

Per-class recognition rate (correct / windows of that class) read from the matrices:

Class (true)	RNN	LSTM	GRU
WALKING	49/75 (65.3%)	62/75 (82.7%)	75/75 (100.0%)
WALKING_UPSTAIRS	53/67 (79.1%)	65/67 (97.0%)	66/67 (98.5%)
WALKING_DOWNSTAIRS	45/62 (72.6%)	61/62 (98.4%)	62/62 (100.0%)
SITTING	59/78 (75.6%)	61/78 (78.2%)	63/78 (80.8%)
STANDING	74/83 (89.2%)	75/83 (90.4%)	76/83 (91.6%)
LAYING	85/85 (100.0%)	85/85 (100.0%)	85/85 (100.0%)
Total misclassified	85	41	23

- **Highest recognition rate.** LAYING is recognized perfectly by all three models (85/85). The GRU is also perfect on WALKING (75/75) and WALKING_DOWNSTAIRS (62/62).
- **Most misclassifications.** For the RNN it is WALKING (26 windows wrong, only 65.3% recall); for the LSTM and GRU it is SITTING (17 and 15 windows wrong, 78.2% and 80.8% recall).
- **Frequently confused pairs.** SITTING ↔ STANDING appears in *all three* matrices (RNN 12 and 7, LSTM 17 and 8, GRU 15 and 7), always with SITTING → STANDING being the larger direction. The RNN additionally confuses the three walking types among themselves: WALKING → UPSTAIRS 20, DOWNSTAIRS → UPSTAIRS 17, UPSTAIRS → DOWNSTAIRS 12. The LSTM has a small scatter of WALKING windows sent to other classes (3, 3, 3 and 4 windows to UPSTAIRS, DOWNSTAIRS, SITTING and STANDING).

- **Consistency across models.** Only partly. The SITTING/STANDING error is consistent, whereas the walking-variant confusion is specific to the RNN; the GRU makes only one error among the dynamic classes (one WALKING_UPSTAIRS window predicted as WALKING).
- **Why.** Plot 1's table shows that SITTING and STANDING have almost the same (near-zero) motion and differ only by a small static gravity offset, so there is very little temporal pattern for a recurrent network to exploit, and extra recurrent capacity (LSTM/GRU) does not remove that error much. The RNN's walking confusions mean it cannot separate gaits that differ in subtle amplitude/shape, which the gated models do.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time	48.7 s	96.3 s	144.6 s
Test F1-score	80.43%	91.13%	95.18%

Plot 5: Model Performance Comparison

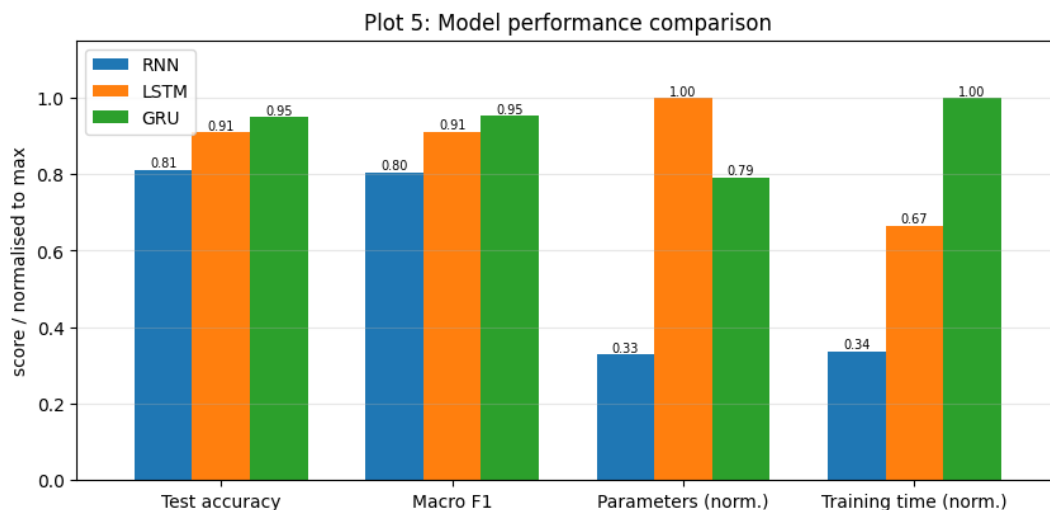
Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

Required inference:

Students should discuss the trade-off among:

Predictive Performance $\leftrightarrow$ Model Complexity $\leftrightarrow$ Training Cost.

Results: Plot 5 and inference



Plot 5: Test accuracy, macro F1 and normalized parameter count and training time (each normalized to the largest of the three models).

Ratio	LSTM / RNN	GRU / RNN	GRU / LSTM
Parameters	3.04	2.41	0.79
Training time	1.98	2.97	1.50

Inference (performance $\leftrightarrow$ complexity $\leftrightarrow$ cost).

- The vanilla RNN is by far the cheapest (1,974 parameters, 48.7 s) but is about 10.7 F1 points behind the LSTM and 14.8 behind the GRU.
- The LSTM costs 3× the RNN's parameters and 2× its time and buys about +10.7 F1 points.
- The GRU had the best F1 (95.18%) with 21% fewer parameters than the LSTM (4,758 versus 6,006). However, in my CPU runs it was also the *slowest* to train (144.6 s versus 96.3 s), so the usual expectation that the GRU trains faster than the LSTM did *not* show up in my timing. I did not profile this; I only note it, and the timing may depend on the CPU implementation of the layers, so this comparison of time is weaker than the comparison of parameters.
- So the best predictive model here (GRU) is not the cheapest in either parameters or time, and the cheapest (RNN) is the worst predictor: performance, complexity and cost do not move together, and the gating mechanism is what pays for itself. Because each model was trained once on a 450-window test set, I would treat the GRU-vs-LSTM gap as suggestive rather than proven.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	77.47	91.11	92.91
64	74.53	94.68	94.37
128	80.43	91.13	95.18

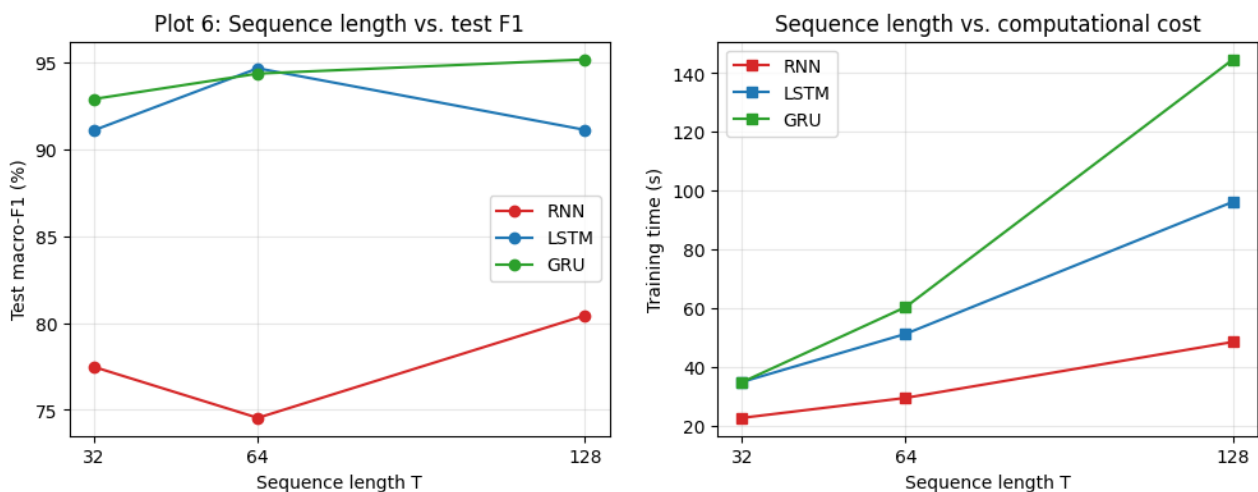
Plot 6: Sequence Length vs. Test F1-score

Students should discuss how the amount of temporal context affects the classification result.

Results: Plot 6 and inference

Inputs were truncated to the *first* T time steps ($T = 32, 64, 128$) with the same normalization, seed and hyper-parameters. Test macro-F1 is in the table above; training times were:

Sequence Length	RNN time (s)	LSTM time (s)	GRU time (s)
32	22.9	35.1	35.0
64	29.7	51.4	60.5
128	48.7	96.3	144.6



Plot 6: Sequence length versus test macro-F1 (left) and versus training time (right).

Inference.

- **Accuracy vs. context.** The effect is small and not monotonic. The GRU improves steadily ($92.91 \rightarrow 94.37 \rightarrow 95.18$, +2.27 points). The LSTM peaks at $T = 64$ (94.68) and ends where it started (91.11 at $T = 32$, 91.13 at $T = 128$). The RNN dips at $T = 64$ (74.53) and is best at $T = 128$ (80.43, +2.97 over $T = 32$).
- **Why.** HAR windows are sampled at 50 Hz, so 32 steps is only about 0.64 s, yet LSTM and GRU already exceed 91% F1 there: the split between dynamic and static activities is visible in a short piece of signal, and the extra context mostly helps the harder cases. The non-monotonic curves (e.g. the LSTM at $T = 128$) are within what I would expect from run-to-run variation with one seed and 450 test windows (standard error of roughly 1.3 accuracy points at 91%), so I do not read them as real effects.
- **Cost.** Quadrupling the length from 32 to 128 multiplied training time by $2.1\times$ (RNN), $2.7\times$ (LSTM) and $4.1\times$ (GRU): the sequential recurrence makes cost grow at least in proportion to T (this also answers Additional Exercise 5).
- **Conclusion.** More temporal context helps modestly or not at all on this dataset for the gated models, but always costs more compute; the RNN remains far below the LSTM/GRU at every length.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

Predicted class : Basketball
 Actual class : Basketball
 Confidence : 0.xx

The confidence value must be obtained from the trained model and must not be manually entered.

Results: predictions with model confidence

These are six test videos, run through the trained CNN–LSTM. The confidence is the maximum softmax probability printed by the program, not typed in.

Video	Predicted class	Actual class	Confidence	Correct
v_TennisSwing_g01_c06	JumpRope	TennisSwing	0.83	no
v_TennisSwing_g05_c03	Basketball	TennisSwing	0.94	no
v_JumpRope_g07_c02	JumpRope	JumpRope	0.93	yes
v_JumpRope_g02_c04	JumpRope	JumpRope	0.99	yes
v_WalkingWithDog_g02_c03	WalkingWithDog	WalkingWithDog	0.99	yes
v_Basketball_g03_c03	Basketball	Basketball	0.98	yes

The two mistakes are made with confidence 0.83 and 0.94, so the softmax confidence is not a reliable warning of an error: the model is confidently wrong on TennisSwing.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

Results: feature dimension, tensor shapes and data used

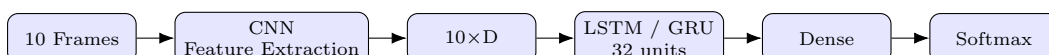
- CNN: MobileNetV2 (ImageNet weights, classification head removed, global average pooling, all layers frozen).
- **CNN feature dimension** $D = 1280$.
- Tensor supplied to the recurrent network per video: $(10, 1280)$; for a batch of B videos: $(B, 10, 1280)$.
- Split shapes: Train $(178, 10, 1280)$, Validation $(37, 10, 1280)$, Test $(85, 10, 1280)$.

Classes. UCF101 has no plain “Running” class and its walking class is **WalkingWithDog**, so I used five classes: Basketball, Biking, WalkingWithDog, TennisSwing and JumpRope, with 60 videos per class (300 in total, picked evenly across the clip groups). I split by *group* (the **gXX** part of the file name; test groups 1–7, validation groups 23–25, training the rest) so that clips cut from the same original video never appear in both train and test.

Class	Train	Validation	Test
Basketball	36	8	16
Biking	36	7	17
WalkingWithDog	35	7	18
TennisSwing	35	7	18
JumpRope	36	8	16
Total	178	37	85

21. CNN–LSTM / CNN–GRU Model

Use the following architecture:

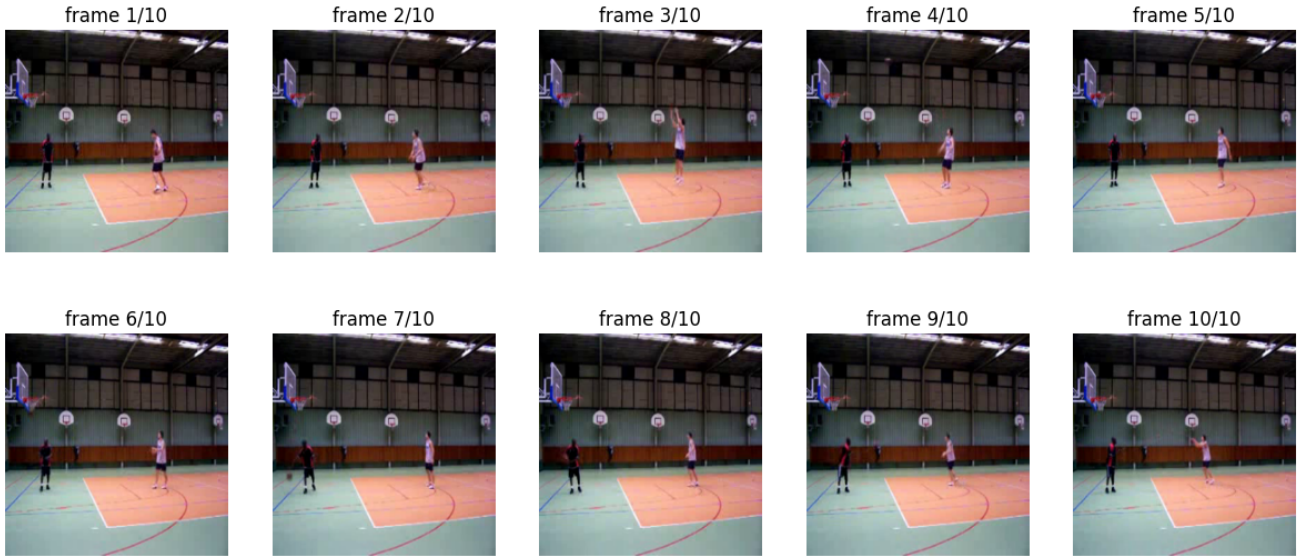


Plot 7: Video Sample Frames

Display a representative sequence of sampled frames from one video.

Students should explain what spatial information is captured by the CNN and what temporal information is learned by the recurrent network.

Plot 7: 10 uniformly sampled frames from one video (Basketball)



Plot 7: Ten uniformly sampled frames (224×224) from one Basketball video.

What each part captures. From each single frame the frozen CNN captures *spatial* content: the court, hoop, the two players, the ball, the colors and textures, i.e. what is in the picture and roughly where. It cannot tell from one frame whether a shot is being taken. The recurrent network sees the ten 1280-dimensional vectors *in order* and learns *temporal* information: the player raising his arms, the ball leaving the hands, the change between frames 2 to 6, and the order in which things happen.

Plot 8: Video Training and Validation Curves

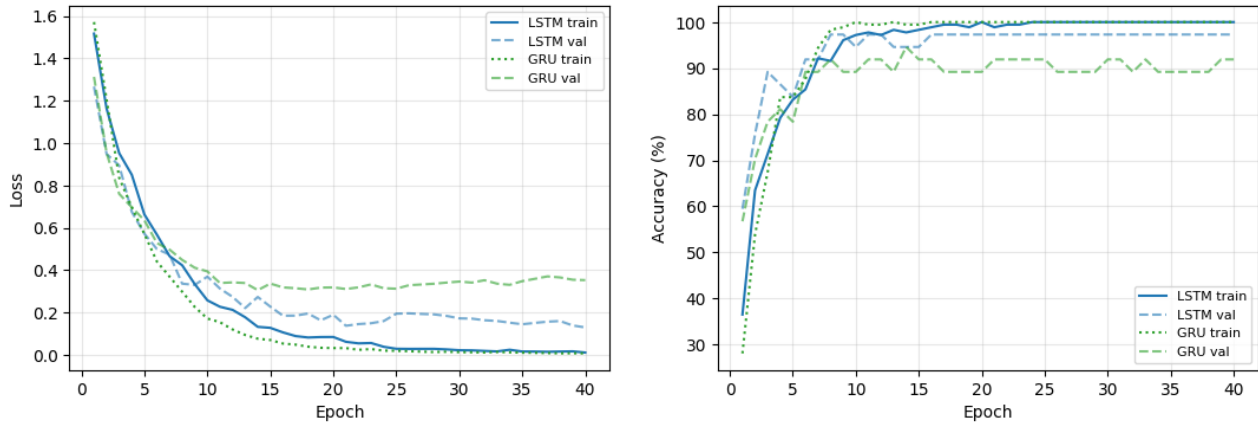
Plot training/validation accuracy and loss for the selected recurrent architecture.

Results: models, Plot 8 and inference

Model: Input (10,1280) → LSTM or GRU (32) → Dropout 0.3 → Dense(16, ReLU) → Dense(5, softmax); Adam 10^{-3} , batch 16, 40 epochs. Trainable parameters: CNN-LSTM $168,064 + 528 + 85 = 168,677$; CNN-GRU $126,144 + 528 + 85 = 126,757$. I picked the architecture using **validation** accuracy only: CNN-LSTM (97.3% versus 91.9% for CNN-GRU).

Model	Val acc (%)	Test acc (%)	Macro P (%)	Macro R (%)	Macro F1 (%)	Params	Train time (s)
CNN-LSTM (selected)	97.3	74.12	74.11	74.17	73.02	168,677	12.5
CNN-GRU	91.9	71.76	73.01	71.95	71.63	126,757	12.9

Plot 8: Video model training curves (selected: CNN-LSTM)



Plot 8: Video-model training and validation loss (left) and accuracy (right); the CNN-LSTM (selected) is drawn solid, the CNN-GRU dotted.

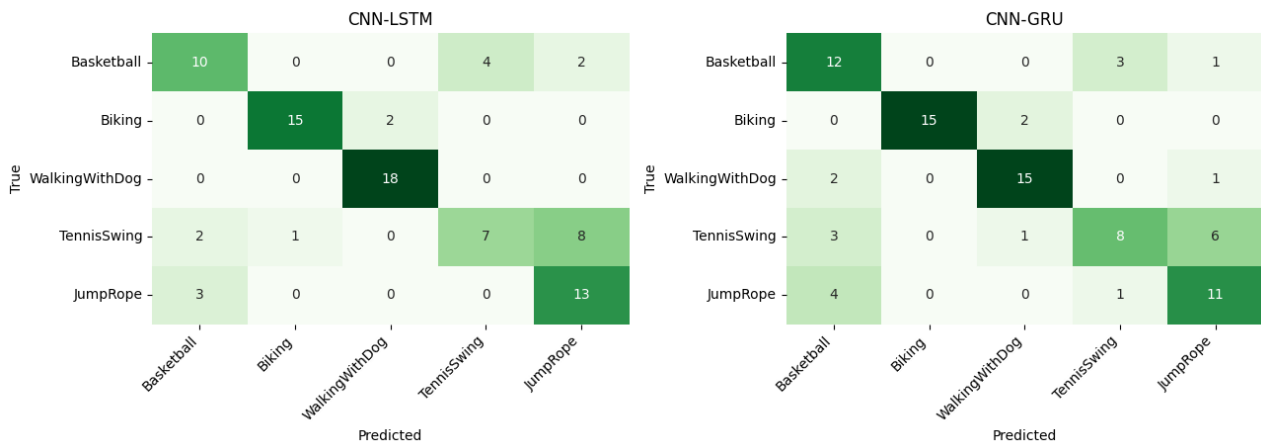
Inference. Training accuracy reaches 100% by about epoch 20 (LSTM) and epoch 10 (GRU) and the training loss keeps falling towards 0.01, while validation loss flattens at about 0.13 (LSTM) and 0.35 (GRU) and validation accuracy stays at 97.3% and 91.9%. On the test videos the accuracy is only 74.1% and 71.8%, so the gap between training (100%) and test accuracy is about 26 to 28 points. My conclusion is that both video models **overfit**: 178 training videos and about 127k–169k trainable parameters on 1280-dimensional inputs is easy to memorize. (The notebook’s automatic check called the CNN-LSTM a “good fit” because it compared only train and validation; the test result shows that is too optimistic.) The validation set is also tiny (37 videos, so one video is worth 2.7 points) and is drawn from different clip groups than the test set, which is likely why validation accuracy (97.3%) is so much higher than test accuracy (74.1%). The 2.4-point difference between LSTM and GRU on test is 2 videos out of 85 (63 versus 61 correct), so I cannot claim either is better.

Plot 9: Video Confusion Matrix

Analyze class-level recognition and temporal confusions.

Results: Plot 9 and inference

Plot 9: Video confusion matrices (test set)



Plot 9: Confusion matrices of the CNN-LSTM (left) and CNN-GRU (right) on the 85 test videos.

Class (true)	CNN-LSTM	CNN-GRU
Basketball	10/16 (62.5%)	12/16 (75.0%)
Biking	15/17 (88.2%)	15/17 (88.2%)
WalkingWithDog	18/18 (100.0%)	15/18 (83.3%)
TennisSwing	7/18 (38.9%)	8/18 (44.4%)
JumpRope	13/16 (81.2%)	11/16 (68.8%)
Total misclassified	22	24

- **Best recognized:** for the CNN-LSTM it is WalkingWithDog (18/18, 100%); for the CNN-GRU it is Biking (15/17, 88.2%), which is also 88.2% for the CNN-LSTM.
- **Worst recognized:** TennisSwing in both models (38.9% and 44.4%). The main confusion is TennisSwing $\rightarrow$ JumpRope (8 videos for the LSTM, 6 for the GRU); the LSTM also sends 4 Basketball videos to TennisSwing.
- **Why (my interpretation, not tested):** TennisSwing and JumpRope are both a single person making a repeated arm/body motion, and TennisSwing needs the *motion* of the swing, which ten frames sampled across the whole clip may catch only partly. The frozen ImageNet features describe objects and scenes well (dog, bike, court), which is why WalkingWithDog and Biking are recognized best. This suggests the temporal part of the model is not doing as much work as the spatial part in this experiment.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

Setup used

Vocabulary: digits 1–9, with 0 used as the start token for the decoder. 12,000 random sequences of length 6 (split 70/15/15 into 8,400 / 1,800 / 1,800), target = the reversed input. Encoder: embedding(32) $\rightarrow$ LSTM(128), whose final states (h, c) are the context. Decoder: embedding(32) $\rightarrow$ LSTM(128) initialised with the context $\rightarrow$ Dense(10, softmax). Training uses **teacher forcing** (the true previous token is the decoder input); at test time I decode greedily, feeding back the model's own previous prediction. Adam 10^{-3} , batch 64, 40 epochs, sparse categorical cross-entropy; 166,794 trainable parameters.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

$$[5, 1, 8, 3].$$

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output	Equals reversed input?
1	[2, 8, 9, 8, 7, 2]	[2, 7, 8, 9, 8, 2]	yes
2	[7, 9, 1, 8, 9, 9]	[9, 9, 8, 1, 9, 7]	yes
3	[9, 1, 1, 5, 1, 4]	[4, 1, 5, 1, 1, 9]	yes
4	[7, 3, 7, 4, 6, 5]	[5, 6, 4, 7, 3, 7]	yes
5	[7, 2, 2, 5, 4, 7]	[7, 4, 5, 2, 2, 7]	yes
6	[5, 8, 8, 5, 8, 9]	[9, 8, 5, 8, 8, 5]	yes
7	[6, 9, 3, 7, 9, 9]	[9, 9, 7, 3, 9, 6]	yes

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

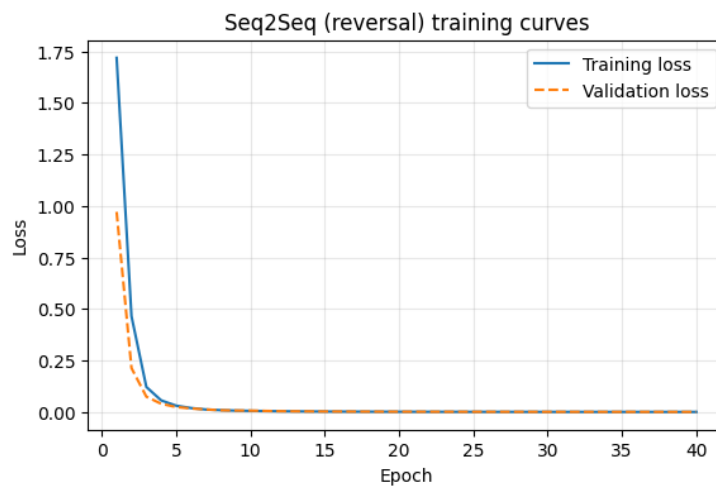
$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

Results: metrics, curves and inference

Metric (test set, 1,800 sequences)	Value
Token accuracy	100.00%
Sequence accuracy	100.00%
Training loss (final epoch)	0.0002
Validation loss (final epoch)	0.0002
Test loss (teacher forcing)	0.0003
Trainable parameters	166,794



Seq2Seq (reversal): training and validation loss per epoch.

The loss drops from about 1.7 (training) at epoch 1 to below 0.01 by epoch 10, and the validation loss follows it closely (no overfitting), so the encoder–decoder learned the reversal completely: all seven displayed examples, and the whole test set, are exactly right.

Inference: token accuracy versus sequence accuracy. In *my* run both are 100.00%, so the gap this question asks about did not appear, because the task is easy. The explanation in general is that a sequence counts as correct only if *all* L tokens are correct. If each token is right with probability p and errors were independent, sequence accuracy is about p^L . For $L = 6$: $p = 0.99$ gives $0.99^6 \approx 0.94$ and $p = 0.95$ gives $0.95^6 \approx 0.74$ (illustrative numbers, not measured), so a model that gets 95% of the tokens right gets only about three quarters of the sequences fully right. In greedy decoding an early wrong token is also fed back into the decoder and can corrupt the tokens after it, so errors are not independent and long sequences suffer even more. Hence sequence accuracy is always at most token accuracy and falls quickly as the sequence length grows.

25. Consolidated Results

Students must complete the following table.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	81.11	82.47	80.30	80.43	1,974
LSTM	90.89	91.54	91.11	91.13	6,006
GRU	94.89	95.37	95.14	95.18	4,758
CNN-LSTM (selected)	74.12	74.11	74.17	73.02	168,677
CNN-GRU	71.76	73.01	71.95	71.63	126,757

A separate table must be provided for the sequence-to-sequence task.

All values in % (precision, recall and F1 are macro averages) and are test-set results. RNN, LSTM and GRU are HAR sensor sequences (6 classes, 450 test windows); the CNN rows are UCF101 videos (5 classes, 85 test videos), so the two groups are not directly comparable.

Sequence-to-sequence (reversal) table:

Model	Token acc (%)	Sequence acc (%)	Train loss	Val loss	Test loss	Parameters
Encoder-decoder LSTM (128)	100.00	100.00	0.0002	0.0002	0.0003	166,794

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

Summary of the mandatory inferences

- **Temporal pattern of the sensor signals.** Walking classes are periodic with large swings (temporal std of body acceleration 0.17–0.23); SITTING, STANDING and LAYING are nearly flat (about 0.01); LAYING differs by its gravity offset (total_acc_x about 0.07). Order matters because the pattern is the periodic sequence itself.
- **Convergence of the RNN.** Slow and unstable: 82.0% validation accuracy after 30 epochs, still rising, with a spike at about epoch 24; never reached 90%.
- **Convergence of the LSTM.** 90% validation accuracy at epoch 18, final 91.8% train / 90.9% validation, with a small bump at epoch 28.
- **Convergence of the GRU.** Fastest: 93.6% validation accuracy at epoch 10, flat loss at the end, final 95.3% train / 95.6% validation.
- **Overfitting/underfitting.** HAR models: no real overfitting (gaps ≤ 0.9 points); the RNN underfits (training accuracy only 82%). Video models: clear overfitting (100% training versus 74.1% / 71.8% test accuracy).
- **Activity-wise errors.** LAYING is always perfect; SITTING $\leftrightarrow$ STANDING is the confusion shared by all three models (SITTING is the worst class for LSTM and GRU); the RNN also confuses the three walking types (WALKING is its worst class, 65.3% recall).
- **Effect of sequence length.** Small and not monotonic (GRU +2.27 points from $T = 32$ to 128, LSTM +0.02, RNN +2.97); cost grows $2.1\times$ to $4.1\times$ for $4\times$ the length.

- **Parameter and computational differences.** Parameters 1,974 (RNN), 6,006 (LSTM), 4,758 (GRU); LSTM = $3.04\times$ and GRU = $2.41\times$ the RNN, GRU = $0.79\times$ LSTM. In my CPU run training time was 48.7 s, 96.3 s and 144.6 s, so the GRU was not faster than the LSTM here.
- **Role of the CNN in video understanding.** It converts each $224\times 224\times 3$ frame into a 1280-dimensional spatial descriptor (objects, scene, appearance) without any training on video, which makes a small video experiment possible.
- **Role of the recurrent model.** It reads the ten feature vectors in order and models how the scene changes, which a per-frame CNN alone cannot do. In my results this part was weak (test accuracy 74%), with TennisSwing confused with JumpRope, showing that with 178 training videos most of the performance comes from appearance rather than motion.
- **Token vs. sequence accuracy.** Sequence accuracy needs every token correct ($\approx p^L$ for independent errors), so it is at most the token accuracy; in my run both were 100% because the reversal task is simple.

Overall conclusion

On the HAR data the gated models clearly beat the vanilla RNN (macro-F1 95.18% GRU, 91.13% LSTM, 80.43% RNN) and converge faster, at $2.4\text{--}3.0\times$ the parameters; the remaining errors are almost all SITTING versus STANDING. A frozen MobileNetV2 with an LSTM/GRU head works as a pipeline, but on only 178 training videos it overfits. The encoder–decoder solves the reversal task perfectly. All numbers come from single runs with seed 42 on a CPU, so small differences (a point or two) should not be over-interpreted.

27. Discussion Questions

1. What is a recurrent neural network?
2. What information is represented by the hidden state?
3. Why is sequence order important?
4. Explain the difference between a feed-forward network and an RNN.
5. Explain Backpropagation Through Time.
6. Why can gradients vanish during BPTT?
7. Why can gradients explode during BPTT?
8. What are long-term dependencies?
9. What is the role of the LSTM cell state?
10. What are the forget, input and output gates in LSTM?
11. What are the reset and update gates in GRU?
12. Compare LSTM and GRU.
13. Why does GRU generally use a simpler gating mechanism than LSTM?
14. Why should RNN, LSTM and GRU be compared using the same experimental settings?
15. What does the confusion matrix reveal that accuracy alone does not?
16. Why should macro F1-score be reported for multi-class classification?
17. What is the significance of sequence length?
18. Why can a CNN be used as a feature extractor for video?
19. What type of information is extracted by the CNN?
20. What type of information is learned by the recurrent network?
21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?
22. Explain the encoder and decoder in sequence-to-sequence learning.
23. What is teacher forcing?
24. Differentiate token accuracy and sequence accuracy.
25. Why must the test set remain untouched during model selection?

Answers

1. A recurrent neural network is a network with a feedback connection that processes a sequence one element at a time, re-using the same weights at every step and carrying a hidden state forward.
2. The hidden state is a compressed summary of everything seen so far $(x_1, \dots, x_t)$; it is used for the output and for computing the next state.
3. The meaning of a sequence depends on the order of its elements. In Plot 1 the gait cycle is the order of rises and falls; shuffling the steps would destroy it.
4. A feed-forward network maps a fixed-size input to an output with no memory and no weight sharing across positions. An RNN has a recurrent loop, keeps a state, shares weights over time and accepts sequences of any length.

5. BPTT unrolls the RNN over T steps into a deep feed-forward graph with shared weights and back-propagates through it. The gradient of each shared weight is the sum of its contributions from every time step.
6. Each step contributes a factor $\text{diag}(1 - h_t^2)W_h$. If its norm is below 1 the product over many steps shrinks exponentially, so early steps get almost no learning signal (Section 8 plot: about 10^{-6} after 60 steps).
7. If the norm of that factor is above 1 (spectral radius of $W_h > 1$) the product grows exponentially (about 10^5 after 60 steps in the plot), making training unstable. Gradient clipping is the usual remedy.
8. Situations where the output at time t depends on inputs many steps earlier.
9. The cell state is a memory line updated additively ($C_t = f_t C_{t-1} + i_t \tilde{C}_t$), so information and gradients can pass through many steps almost unchanged.
10. The forget gate decides how much of the old cell state is kept, the input gate how much of the new candidate is written, and the output gate how much of the cell state is exposed as h_t .
11. The reset gate controls how much of the previous state is used to build the candidate state; the update gate interpolates between the old state and the candidate.
12. The LSTM has three gates and a separate cell state (4 weight blocks); the GRU has two gates and no separate cell (3 blocks), so it is smaller (4,758 versus 6,006 parameters in my model). In my run the GRU also had the higher test F1 (95.18 versus 91.13), but with one run per model I would call them comparable, and the LSTM matched the GRU at 64 units (Additional Exercise 1).
13. The GRU merges the cell and hidden state, ties the forget and input gates into one update gate (z_t and $1 - z_t$) and drops the output gate, which removes a weight block.
14. So that any difference in results can be attributed to the architecture and not to differences in data, optimizer, batch size, epochs or preprocessing.
15. It shows which classes are confused with which (e.g. SITTING $\rightarrow$ STANDING 15–17 times), and per-class recall, which overall accuracy hides.
16. Macro F1 gives every class equal weight and combines precision and recall, so a weak minority class cannot be hidden by strong majority classes.
17. Sequence length is the amount of temporal context the model sees; longer sequences give more information but cost more time (up to $4.1\times$ for $4\times$ the length in my runs) and make gradient flow harder.
18. A CNN pretrained on ImageNet already gives compact, general spatial descriptors of images, so it can be frozen and used for each frame, which needs no video-scale training data and is cheap.
19. Spatial content of a frame: edges, textures, shapes, objects (ball, bike, dog) and scene layout.
20. Temporal information: motion and change across frames and the order of events.
21. A video is an ordered set of frames, so stacking the per-frame vectors as (T, D) gives the recurrent network a time axis to model.
22. The encoder reads the input sequence and compresses it into a context state (h, c) ; the decoder is initialised with this state and generates the output sequence one token at a time.
23. Teacher forcing feeds the decoder the true previous token during training instead of its own prediction. It speeds up and stabilizes training but creates a train/test mismatch (exposure bias) because at test time the decoder must use its own outputs.
24. Token accuracy is the fraction of correct tokens; sequence accuracy is the fraction of sequences with every token correct ($\approx p^L$ under independent errors), so it is never higher.
25. The test set must give an unbiased estimate of performance on unseen data. If I choose models or hyper-parameters using it, information leaks and the reported score becomes over-optimistic.

28. Additional Exercises

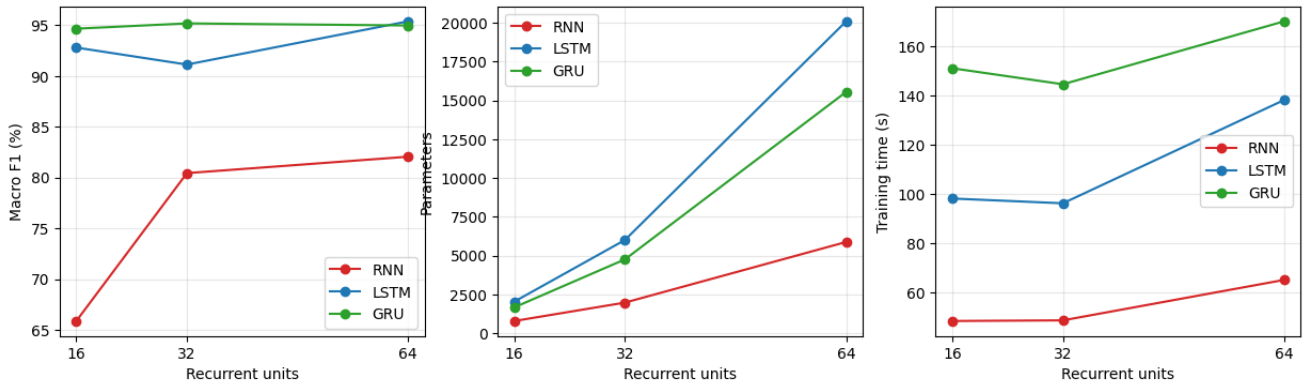
1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.
2. Compare GRU and LSTM using the same number of recurrent units.
3. Add a second recurrent layer and investigate whether performance improves.
4. Compare a bidirectional LSTM with a unidirectional LSTM.
5. Change the sequence length and study the effect on computational cost.
6. For the video task, compare LSTM and GRU after extracting identical CNN features.
7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.

Results of the additional exercises

Exercises 1 and 2: 16, 32 and 64 recurrent units (all three models, same protocol).

Model	Units	Accuracy (%)	Macro F1 (%)	Parameters	Training time (s)
RNN	16	68.00	65.87	790	48.4
RNN	32	81.11	80.43	1,974	48.7
RNN	64	82.67	82.05	5,878	65.1
LSTM	16	92.67	92.80	2,038	98.2
LSTM	32	90.89	91.13	6,006	96.3
LSTM	64	95.11	95.38	20,086	138.3
GRU	16	94.44	94.66	1,670	151.1
GRU	32	94.89	95.18	4,758	144.6
GRU	64	94.67	94.98	15,542	170.2

Exercise 1/2: effect of recurrent units



Exercises 1 and 2: effect of the number of recurrent units on macro-F1, parameter count and training time.

- The RNN is capacity-limited: F1 rises from 65.87 to 80.43 to 82.05 as units go $16 \rightarrow 32 \rightarrow 64$. The GRU is flat (94.66, 95.18, 94.98), so 16 units and 1,670 parameters are already enough for it. The LSTM is non-monotonic (92.80, 91.13, 95.38); the dip at 32 units is most likely run-to-run noise, not a real effect.
- **GRU vs. LSTM at equal units** (macro F1): GRU is ahead at 16 units (94.66 vs 92.80) and at 32 (95.18 vs 91.13); the LSTM is marginally ahead at 64 (95.38 vs 94.98, only 0.4 points, within noise). So the GRU reaches its best result with fewer parameters, and the LSTM needs more units to catch up.
- Parameters grow faster than linearly because the recurrent weights scale as $\text{units} \times (\text{inputs} + \text{units})$: from 32 to 64 units the LSTM goes from 6,006 to 20,086 ($3.3\times$). Training time only changed noticeably at 64 units (LSTM +44%, RNN +34%, GRU +18% over 32 units).

Exercise 3: second recurrent layer (two stacked layers of 32 units).

Model	F1 1-layer (%)	F1 2-layer (%)	Params 1-layer	Params 2-layer	Time 1-layer (s)	Time 2-layer (s)
RNN	80.43	67.76	1,974	4,054	48.7	122.1
LSTM	91.13	95.18	6,006	14,326	96.3	308.8
GRU	95.18	95.61	4,758	11,094	144.6	254.7

A second layer clearly **helped the LSTM** (+4.05 F1 points), was **useless for the GRU** (+0.43, within noise, for $1.8\times$ the time) and **hurt the RNN** (-12.7 points): its training became unstable (validation accuracy went from 84.0% at epoch 20 down to 72.0% at epoch 30), which fits the difficulty of passing gradients through a deeper stack of plain recurrent layers.

Exercise 4: bidirectional LSTM versus unidirectional LSTM (32 units).

Model	Accuracy (%)	Macro F1 (%)	Parameters	Training time (s)
LSTM (unidirectional)	90.89	91.13	6,006	96.3
BiLSTM	94.89	95.18	11,894	103.9

The BiLSTM improved macro-F1 by 4.05 points (18 more windows correct) with about twice the parameters ($1.98\times$) but only 8% more time. Since the whole window is available before classification, reading it in both directions is allowed here, and each direction ends with a different part of the window fresh in its state; I expect that to be the reason, but I did not test it.

Exercise 5: sequence length versus computational cost. See Section 17: for $T = 32 \rightarrow 128$, training time grew from 22.9 to 48.7 s (RNN, $2.1\times$), 35.1 to 96.3 s (LSTM, $2.7\times$) and 35.0 to 144.6 s (GRU, $4.1\times$).

Exercise 6: LSTM versus GRU on identical CNN features. Same extracted features, same split: CNN–LSTM 74.12% accuracy / 73.02% F1 / 168,677 parameters / 12.5 s; CNN–GRU 71.76% / 71.63% / 126,757 parameters (25% fewer) / 12.9 s. The difference is 2 test videos out of 85 (63 versus 61 correct), so there is no clear winner; the GRU is simply smaller.

Exercise 7: seq2seq with a different output length. I used input length 6 and output length 3 (the target is the first three elements of the reversed input, e.g. $[2, 8, 9, 8, 7, 2] \rightarrow [2, 7, 8]$). Result: token accuracy 100.00%, sequence accuracy 100.00%, training loss 0.0001, validation loss 0.0001, test loss 0.0001. The encoder–decoder does not need equal lengths, because the decoder simply generates as many tokens as it is asked to (3).

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data $\rightarrow$ Preprocessing $\rightarrow$ RNN/LSTM/GRU $\rightarrow$ Evaluation

and the video-understanding pipeline:

Video $\rightarrow$ Frames $\rightarrow$ CNN Features $\rightarrow$ LSTM/GRU $\rightarrow$ Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence $\rightarrow$ Encoder $\rightarrow$ Context $\rightarrow$ Decoder $\rightarrow$ Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khuram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>